

Tema: Automatizarea curățării fișierelor temporare folosind Shell Script

1. Scopul lucrării

Scopul lucrării de laborator a fost familiarizarea cu automatizarea unor sarcini repetitive folosind scripturi Shell, precum și utilizarea sistemului de control al versiunilor Git și a platformei GitHub.

Pentru realizarea laboratorului a fost aleasă sarcina **Cleaning up temporary files**, care presupune crearea unui script Bash capabil să elimine automat fișierele cu anumite extensii dintr-un director specificat de utilizator.

Lucrarea a fost realizată pe sistemul de operare **macOS**, folosind aplicația Terminal, Bash, Git și GitHub.

2. Sarcina selectată

A fost aleasă sarcina a doua din lista propusă: **Cleaning up temporary files**.

Scriptul trebuia să respecte următoarele cerințe:

- să se numească `cleanup.sh`;
- să primească drept prim argument calea către directorul care trebuie curățat;
- să accepte opțional extensiile fișierelor care trebuie eliminate;
- dacă nu este specificată nicio extensie, să elimine implicit fișierele cu extensia `.tmp`;
- să verifice dacă directorul introdus există;
- să afișeze mesaje de eroare corespunzătoare;
- la final să afișeze numărul total de fișiere șterse.

3. Crearea repository-ului

Pentru organizarea proiectului a fost creat un repository local numit `automation`.

În Terminal au fost utilizate comenzile:

```
mkdir automation
cd automation
git init
git branch -M main
```

Comanda `git init` a inițializat un nou repository Git, iar branch-ul principal a fost denumit `main`.

În directorul principal a fost creat și fișierul:

`README.md`

Acesta conține o scurtă descriere a repository-ului și a scopului lucrării.

După crearea fișierului README, acesta a fost adăugat în Git și salvat printr-un commit:

```
git add README.md
git commit -m "Initial commit: add repository README"
```

4. Crearea branch-ului **lab01**

Conform cerințelor laboratorului, dezvoltarea scriptului nu a fost realizată direct în branch-ul principal.

A fost creat un branch separat:

```
git checkout -b lab01
```

Prin această comandă a fost creat branch-ul `lab01`, iar lucrul a continuat în cadrul acestuia.

În interiorul repository-ului a fost creat directorul:

```
lab01
```

Structura proiectului a devenit:

```
automation/
├── README.md
└── lab01/
```

5. Crearea scriptului **cleanup.sh**

În folderul `lab01` a fost creat fișierul:

```
cleanup.sh
```

Scriptul realizat este următorul:

```
#!/bin/bash
```

```
# Check if at least one argument was provided
if [ "$#" -lt 1 ]; then
```

```

        echo "Error: Directory path is required."
        echo "Usage: $0 <directory> [extension1]
[extension2] ..."
        exit 1
    fi

# First argument is the directory that will be cleaned
directory="$1"

# Remove the first argument from the argument list
shift

# Check if the directory exists
if [ ! -d "$directory" ]; then
    echo "Error: Directory '$directory' does not exist."
    exit 1
fi

# If no extensions were specified, use .tmp by default
if [ "$#" -eq 0 ]; then
    extensions=".tmp"
else
    extensions=("$@")
fi

deleted_count=0

# Process every specified extension
for extension in "${extensions[@]}; do

    if [[ "$extension" != .* ]]; then
        extension=".$extension"
    fi

    while IFS= read -r -d ' ' file; do
        if rm "$file"; then
            echo "Deleted: $file"
            deleted_count=$((deleted_count + 1))
        fi
    done < <(find "$directory" -type f -name "$extension"
-print0)

done

echo "Deleted files: $deleted_count"

```

Pentru ca scriptul să poată fi executat direct din Terminal, i-au fost acordate drepturi de execuție:

```
chmod +x cleanup.sh
```

6. Principiul de funcționare al scriptului

Prima linie:

```
#!/bin/bash
```

specifică faptul că scriptul trebuie executat folosind interpretorul Bash.

În continuare este verificat numărul de argumente:

```
if [ "$#" -lt 1 ]; then
```

Variabila `$#` reprezintă numărul argumentelor transmise scriptului. Dacă utilizatorul nu introduce niciun argument, este afișat un mesaj de eroare și scriptul se oprește.

Primul argument este salvat în variabila:

```
directory="$1"
```

Acesta reprezintă calea către directorul care trebuie curățat.

După salvarea primului argument este utilizată comanda:

```
shift
```

Aceasta elimină primul argument din lista de argumente, astfel încât argumentele rămase să poată fi interpretate ca extensii.

Scriptul verifică apoi existența directorului:

```
if [ ! -d "$directory" ]; then
```

Dacă directorul nu există, este afișat un mesaj corespunzător și execuția se încheie.

Dacă utilizatorul nu specifică o extensie, este utilizată automat extensia:

```
.tmp
```

prin secvența:

```
if [ "$#" -eq 0 ]; then  
    extensions=( ".tmp" )
```

Pentru fiecare extensie specificată se utilizează comanda `find`, care caută fișierele corespunzătoare:

```
find "$directory" -type f -name "$extension"
```

Opțiunea:

```
-type f
```

indică faptul că trebuie căutate doar fișiere, nu și directoare.

Fișierele găsite sunt șterse cu ajutorul comenzii:

```
rm
```

Pentru fiecare fișier eliminat, variabila `deleted_count` este incrementată.

La sfârșitul executării este afișat numărul total de fișiere șterse:

```
echo "Deleted files: $deleted_count"
```

7. Testarea scriptului

Pentru testarea funcționalității a fost creat un director separat pe Desktop:

```
mkdir -p ~/Desktop/cleanup_test
```

În acest director au fost create mai multe fișiere:

```
touch ~/Desktop/cleanup_test/file1.tmp  
touch ~/Desktop/cleanup_test/file2.tmp  
touch ~/Desktop/cleanup_test/file3.log  
touch ~/Desktop/cleanup_test/document.txt
```

Conținutul directorului înainte de rularea scriptului a fost:

```
file1.tmp  
file2.tmp  
file3.log  
document.txt
```

Scriptul a fost executat fără specificarea extensiei:

```
./cleanup.sh ~/Desktop/cleanup_test
```

Deoarece nu a fost transmisă nicio extensie, scriptul a utilizat implicit extensia `.tmp`.

Au fost șterse:

```
file1.tmp  
file2.tmp
```

iar fișierele:

```
file3.log  
document.txt
```

au rămas neschimbate.

La final a fost afișat:

```
Deleted files: 2
```

Astfel a fost verificată funcționarea extensiei implicite.

8. Testarea cu o extensie specificată

Scriptul a fost testat și cu extensia `.log`.

Comanda utilizată a fost:

```
./cleanup.sh ~/Desktop/cleanup_test .log
```

În acest caz, au fost eliminate doar fișierele cu extensia `.log`.

Scriptul permite și specificarea mai multor extensii în aceeași comandă:

```
./cleanup.sh ~/Desktop/cleanup_test .tmp .log .cache
```

În acest caz sunt eliminate toate fișierele care corespund extensiilor specificate.

De asemenea, scriptul permite introducerea extensiei fără punct:

```
./cleanup.sh ~/Desktop/cleanup_test tmp log
```

Scriptul adaugă automat caracterul `.` înaintea extensiei.

9. Verificarea tratării erorilor

A fost verificat și cazul în care utilizatorul introduce un director inexistent.

De exemplu:

```
./cleanup.sh ~/Desktop/folder_inexistent
```

Scriptul afișează un mesaj de forma:

```
Error: Directory '/Users/.../Desktop/folder_inexistent' does  
not exist.
```

De asemenea, dacă scriptul este executat fără niciun argument:

```
./cleanup.sh
```

este afișată informația despre modul corect de utilizare:

```
Error: Directory path is required.  
Usage: ./cleanup.sh <directory> [extension1] [extension2] ...
```

Prin aceste verificări s-a asigurat că scriptul gestionează corect situațiile de eroare.

10. Crearea documentației

În folderul `lab01` a fost creat un al doilea fișier:

```
README.md
```

Acesta conține:

- descrierea scriptului;
- cerințele necesare pentru rulare;
- sintaxa de utilizare;
- exemple de comenzi;
- informații despre tratarea erorilor;
- exemple ale rezultatului afișat.

Structura finală a proiectului a devenit:

```
automation/
├── README.md
└── lab01/
    ├── README.md
    └── cleanup.sh
```

11. Salvarea modificărilor în Git

După finalizarea scriptului și a documentației, fișierele au fost adăugate în Git:

```
git add lab01
```

Starea repository-ului a fost verificată cu:

```
git status
```

După aceea a fost creat un commit:

```
git commit -m "Lab 01: add temporary file cleanup script"
```

Prin această etapă, toate modificările realizate în branch-ul `lab01` au fost salvate în istoricul Git.

12. Publicarea proiectului pe GitHub

Pe GitHub a fost creat repository-ul `Automation`.

Repository-ul local a fost conectat cu repository-ul remote prin:

```
git remote add origin https://github.com/USERNAME/
Automation.git
```

Pentru verificarea conexiunii a fost utilizată comanda:

```
git remote -v
```

Branch-ul principal a fost publicat pe GitHub:

```
git push -u origin main
```

Apoi a fost publicat și branch-ul laboratorului:

```
git checkout lab01  
git push -u origin lab01
```

În cazul autentificării HTTPS, GitHub nu mai acceptă parola obișnuită a contului pentru operațiile Git. Pentru autentificare a fost utilizat un **Personal Access Token**.

13. Unirea modificărilor cu branch-ul principal

După finalizarea lucrului pe branch-ul `lab01`, s-a revenit pe branch-ul principal:

```
git checkout main
```

Modificările realizate în branch-ul laboratorului au fost integrate în `main`:

```
git merge lab01
```

După merge, modificările finale au fost trimise către GitHub:

```
git push origin main
```

Astfel, branch-ul principal al repository-ului conține versiunea finală a laboratorului.

14. Rezultatul lucrării

În urma realizării laboratorului a fost creat un script Shell care automatizează procesul de eliminare a fișierelor temporare.

Scriptul:

- acceptă calea unui director;
- verifică existența directorului;
- șterge implicit fișiere `.tmp`;
- permite alegerea uneia sau mai multor extensii;
- afișează fiecare fișier șters;

- numără fișierele eliminate;
- afișează rezultatul la final;
- tratează cazurile de eroare.

Proiectul a fost organizat cu ajutorul Git, dezvoltat într-un branch separat și ulterior integrat în branch-ul principal. Versiunea finală a fost publicată într-un repository GitHub.

Concluzie

În cadrul acestei lucrări de laborator am realizat un script Bash pentru automatizarea procesului de curățare a fișierelor temporare. Prin această sarcină am utilizat comenzi specifice sistemelor Unix, precum `find`, `rm`, `chmod`, `touch` și `mkdir`, și am lucrat cu argumente, variabile, condiții, bucle și verificarea erorilor în Bash.

De asemenea, am utilizat Git pentru versionarea proiectului, am creat un branch separat pentru laborator și am realizat integrarea modificărilor în branch-ul principal. Proiectul a fost publicat pe GitHub, ceea ce permite păstrarea și distribuirea codului într-un repository remote.

Lucrarea a demonstrat modul în care un script simplu poate automatiza o sarcină repetitivă și poate reduce timpul necesar administrării manuale a fișierelor.